

Survival Kit : Langage C & Chaînes

L'antisèche ultime pour les partiels - Gestion de la mémoire et manipulation de texte

BASIQUES <string.h>

Opérations classiques. Attention au \0!

- `size_t strlen(s)` : Longueur sans \0.
- `strcpy(dest, src)` : Copie `src` dans `dest`. **Danger** : Buffer overflow si `dest` trop petit.
- `strncpy(dest, src, n)` : Copie max `n` char. **Piège** : Ne met pas de \0 si coupé!
- `strcat(dest, src)` : Ajoute à la fin.
- `strncat(dest, src, n)` : Ajout sécurisé.
- `strcmp(s1, s2)` : Compare.
 - 0 : Égales.
 - <0 : s1 avant s2.
 - >0 : s1 après s2.
- `strdup(s)` : Malloc + Copie + Return. **Impératif** : Faire un `free()` après.

RECHERCHE <string.h>

- `strchr(s, c)` : Pointe vers la 1ère occurrence de `c`. Return NULL si absent.
- `strrchr(s, c)` : Pointe vers la **dernière** occurrence.
- `strstr(haystack, needle)` : Cherche une sous-chaîne dans une chaîne.
- `strtok(str, delim)` : Découpe la chaîne (split).
Note : Destructif (remplace sép. par \0). Utiliser NULL pour continuer.

```
char *token = strtok(str, " ");
while (token) {
    printf("%s\n", token);
    token = strtok(NULL, " ");
}
```

INSPECTION <ctype.h>

Test sur un seul caractère (int c).

- `isalpha(c)` : A-Z ou a-z.
- `isdigit(c)` : 0-9.
- `isalnum(c)` : Lettre OU Chiffre.
- `isspace(c)` : Espace, \t, \n, \v, \f, \r.
- `isprint(c)` : Caractère imprimable.
- `toupper(c)` / `tolower(c)` : Conversion casse.

CONVERSIONS

Bibliothèques <stdlib.h> et <stdio.h>

- `atoi(str)` : String vers Int. *Limité* : Retourne 0 si erreur.
- `strtol(str, &end, base)` : La version Pro. Permet de gérer les erreurs et la base (10, 16, 2...).
- `sprintf(buf, fmt, ...)` : "Print" dans une string.
Ex : `sprintf(buf, "%d", 42);` → Int to Str.
- `sscanf(str, fmt, ...)` : Lire depuis une string.

MÉMOIRE <string.h>

Opérations sur `void*`. Pas d'arrêt au \0.

- `memset(ptr, val, n)` : Remplit `n` octets avec `val`.
Usage : `memset(arr, 0, sizeof(arr));`
- `memcpy(dst, src, n)` : Copie rapide. **Interdit** : Chevauchement (overlap).
- `memmove(dst, src, n)` : Copie sûre (gère l'overlap).
- `memcmp(p1, p2, n)` : Compare deux blocs mémoire.

ALGOS À LA CON (EXAM)

1. Swap sans variable temp

```
void swap(int *a, int *b) {
    *a = *a ^ *b;
    *b = *a ^ *b;
    *a = *a ^ *b;
}
```

2. Inverser une chaîne (in-place)

```
void strrev(char *str) {
    int i = 0, j = strlen(str) - 1;
    while (i < j) {
        char temp = str[i];
        str[i++] = str[j];
        str[j--] = temp;
    }
}
```

3. Est puissance de 2 ?

```
int is_pow2(int n) {
    return (n > 0 && !(n & (n - 1)));
}
```

MEMO POINTEURS

- `char *s = "Bonjour";` : Stocké en lecture seule (RO). `s[0] = 'b'` → **Segfault**.
- `char s[] = "Bonjour";` : Tableau modifiable sur la stack. OK.
- `malloc(n)` : Toujours vérifier si retour != NULL.
- `free(ptr)` : Ne jamais free deux fois (Double Free). Mettre `ptr = NULL` après.

BONUS (POUR BRILLER)

- `strcspn(s, reject)` : Longueur du segment initial qui NE contient PAS les chars de `reject`. *Astuce* : Pour enlever le \n de `fgets` : `s[strcspn(s, "\n")] = 0;`
- `strspn(s, accept)` : Longueur du segment initial contenant QUE des chars de `accept`.
- `perror("msg")` : Affiche l'erreur système actuelle sur `stderr`.